

Embedded Systems Lab Setup & Sensor Interfacing

Embedded Systems Task-2

Project Report on

SMART LIGHTING USING LDR

Submitted By

Panchakarla shanmukha chaitanya

B.Tech – 2nd Year (ECE)

Electronics and Communication Engineering

Submitted To



MainCrafts Technologies

Academic Year: 2025 – 2026

Project Domain:

Embedded Systems & Internet of Things (IoT)

Smart Lighting System Using LDR

The Smart Lighting System using a Light Dependent Resistor (LDR) is an automated lighting solution designed to control a light source based on ambient light intensity. The system uses an Arduino microcontroller to monitor environmental light conditions through an LDR sensor. This project helps reduce energy consumption and demonstrates the application of sensors and embedded systems in automation.

Components Required

Component Name	Quantity	Purpose
Arduino Uno	1	Main microcontroller used to process sensor data and control the LED
Light Dependent Resistor (LDR)	1	Detects ambient light intensity
10 k Ω Resistor	1	Forms a voltage divider circuit with the LDR
LED	1	Represents the lighting system output
220 Ω Resistor	1	Limits current through the LED and protects it from damage
Breadboard	1	Used for temporary circuit connections

Working Principle

The Smart Lighting System operates based on the intensity of ambient light detected by a Light Dependent Resistor (LDR). The resistance of the LDR varies according to the amount of light falling on its surface. When the surrounding environment is bright, the resistance of the LDR decreases, producing a higher analog voltage. When the environment becomes dark, the resistance increases, resulting in a lower analog voltage.

The LDR is connected to the Arduino Uno through a voltage divider circuit. The Arduino continuously reads the analog voltage from the LDR using its analog input pin (A0). The obtained value is compared with a predefined threshold value programmed in the Arduino.

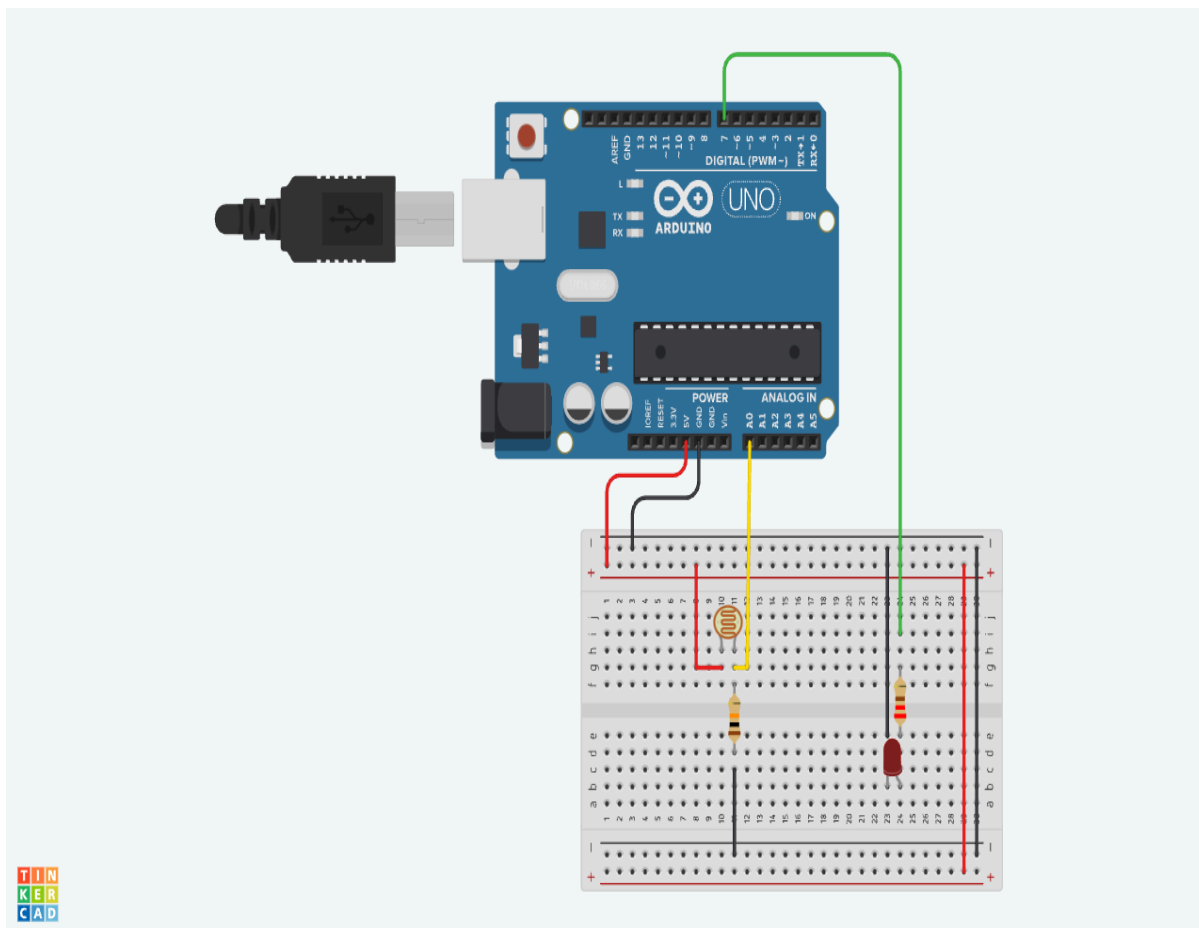
If the measured light intensity is below the threshold value, indicating low-light or dark conditions, the Arduino activates the output pin and turns the LED ON. When the light intensity rises above the threshold value, indicating sufficient brightness, the Arduino deactivates the output pin and turns the LED OFF.

This automatic process continuously repeats, allowing the system to respond to changes in environmental lighting conditions without human intervention. As a result, the system helps conserve energy, improves convenience, and provides an efficient solution for automatic lighting control.

Flow of Operation:

1. LDR senses the ambient light intensity.
2. Arduino reads the sensor value through analog pin A0.
3. The sensor value is compared with the threshold value.
4. If the light intensity is low, the LED turns ON.
5. If the light intensity is high, the LED turns OFF.
6. The process repeats continuously for real-time monitoring and control.

Circuit Diagram



Arduino IDE Code

```
// smart lighting using LDR

const int ldrPin = A0; // LDR sensor connected to Analog pin A0

const int ledPin = 13; // LED or Relay connected to Digital pin 13

int threshold = 500;      // Light threshold (adjust based on your
                           // environment)

void setup() {

  pinMode(ledPin, OUTPUT);

  pinMode(ldrPin, INPUT);

  Serial.begin(9600); // Start serial monitor to read LDR values
}

void loop() {

  int ldrValue = analogRead(ldrPin); // Read the light value (0 to 1023)

  Serial.println(ldrValue);          // Print value to Serial Monitor

  // If it's dark (sensor value drops below threshold)
  if (ldrValue < threshold) {

    digitalWrite(ledPin, HIGH); // Turn the light ON
  }

  // If it's bright
  else {
```

```
    digitalWrite(ledPin, LOW); // Turn the light OFF
}

delay(500); // Wait for half a second before reading again
}
```

Code Explanation

The program is designed to automatically control an LED based on the light intensity detected by an LDR sensor. The keywords `const int` are used to define the LDR pin (`A0`) and LED pin (`13`), while the variable `threshold` stores the reference light value.

In the `setup()` function, `pinMode()` configures the LED as an `OUTPUT` and the LDR as an `INPUT`. The `Serial.begin(9600)` function starts serial communication for monitoring sensor readings.

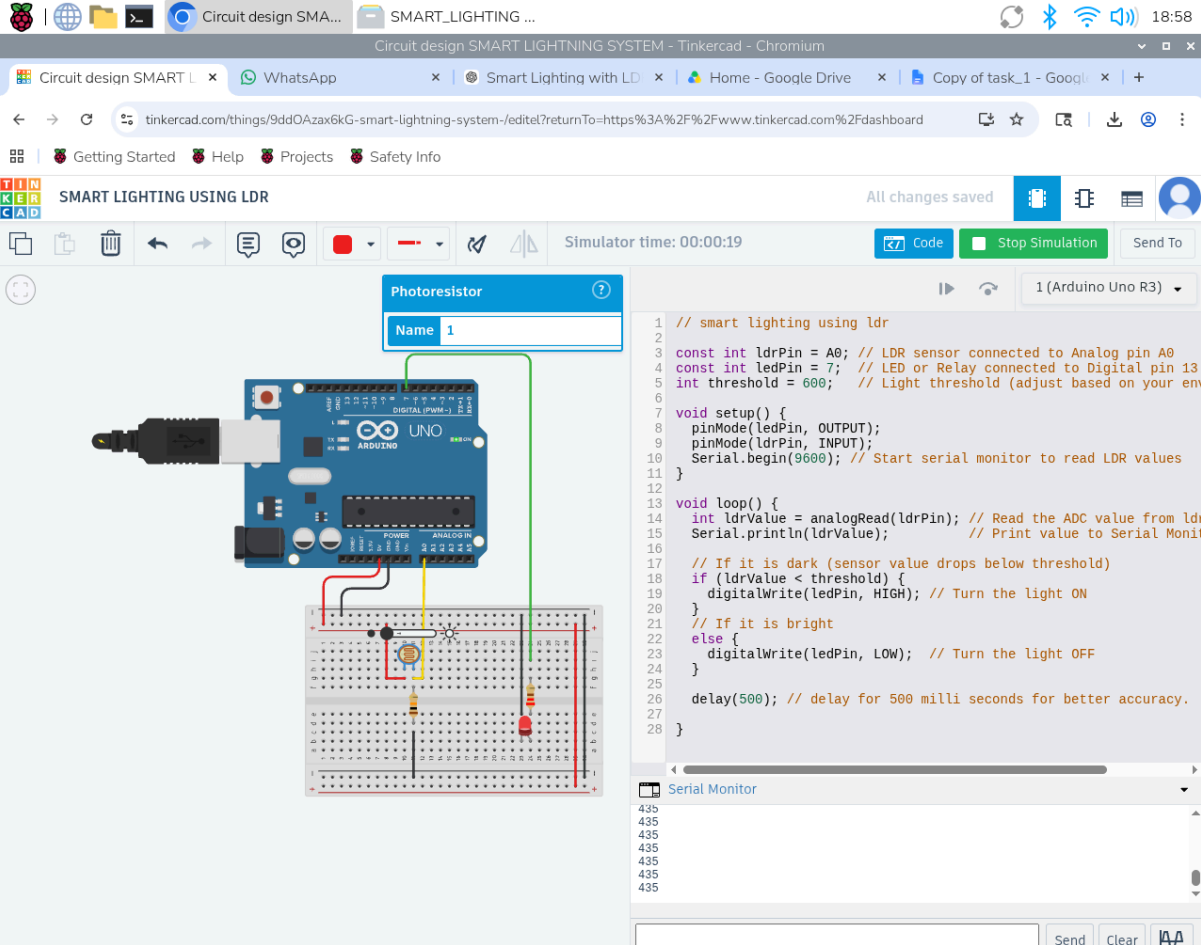
Inside the `loop()` function, `analogRead(ldrPin)` reads the light intensity from the LDR and stores it in `ldrValue`. The value is displayed on the Serial Monitor using `Serial.println()`. An `if-else` statement compares the sensor value with the threshold. If `ldrValue < threshold`, `digitalWrite(ledPin, HIGH)` turns the LED ON. Otherwise, `digitalWrite(ledPin, LOW)` turns the LED OFF. The `delay(500)` function pauses the program for 500 milliseconds before the next reading.

Key Functions Used

- `pinMode()` – Configures pin as INPUT or OUTPUT.
- `analogRead()` – Reads analog value from the LDR.
- `digitalWrite()` – Controls the LED state.
- `Serial.begin()` – Starts serial communication.

- `Serial.println()` – Displays sensor values.
- `if-else` – Makes decisions based on light intensity.
- `delay()` – Creates a time delay between readings.

Simulation Result



The screenshot shows a Tinkercad simulation of a "SMART LIGHTING USING LDR" system. The circuit includes an Arduino Uno connected to a breadboard with a photoresistor and an LED. The photoresistor is connected to analog pin A0, and the LED is connected to digital pin 7. The code on the right implements a logic where the LED turns on when the sensor value is below a threshold of 600 and turns off when it is above the threshold. A 500ms delay is used between readings.

```

1 // smart lighting using ldr
2
3 const int ldrPin = A0; // LDR sensor connected to Analog pin A0
4 const int ledPin = 7; // LED or Relay connected to Digital pin 13
5 int threshold = 600; // Light threshold (adjust based on your env)
6
7 void setup() {
8   pinMode(ledPin, OUTPUT);
9   pinMode(ldrPin, INPUT);
10  Serial.begin(9600); // Start serial monitor to read LDR values
11 }
12
13 void loop() {
14   int ldrValue = analogRead(ldrPin); // Read the ADC value from ldr
15   Serial.println(ldrValue); // Print value to Serial Monitor
16
17   // If it is dark (sensor value drops below threshold)
18   if (ldrValue < threshold) {
19     digitalWrite(ledPin, HIGH); // Turn the light ON
20   }
21   // If it is bright
22   else {
23     digitalWrite(ledPin, LOW); // Turn the light OFF
24   }
25
26   delay(500); // delay for 500 milli seconds for better accuracy.
27
28 }

```

The Serial Monitor shows the sensor values being read, which are currently 435.